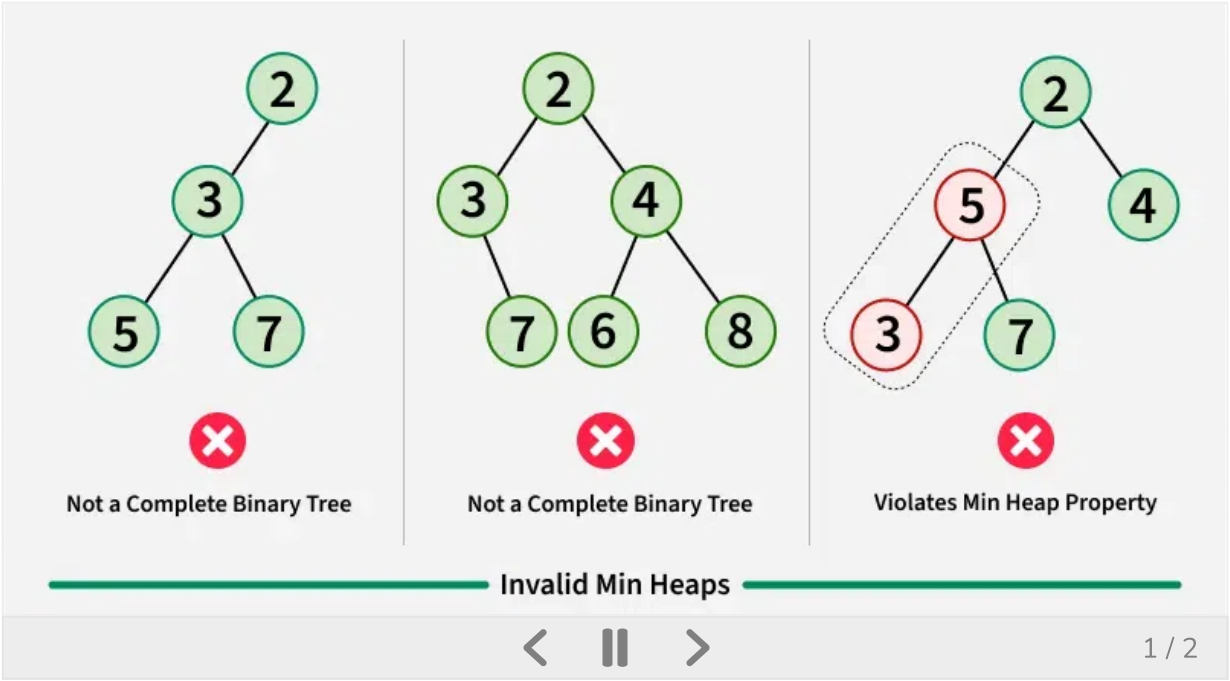
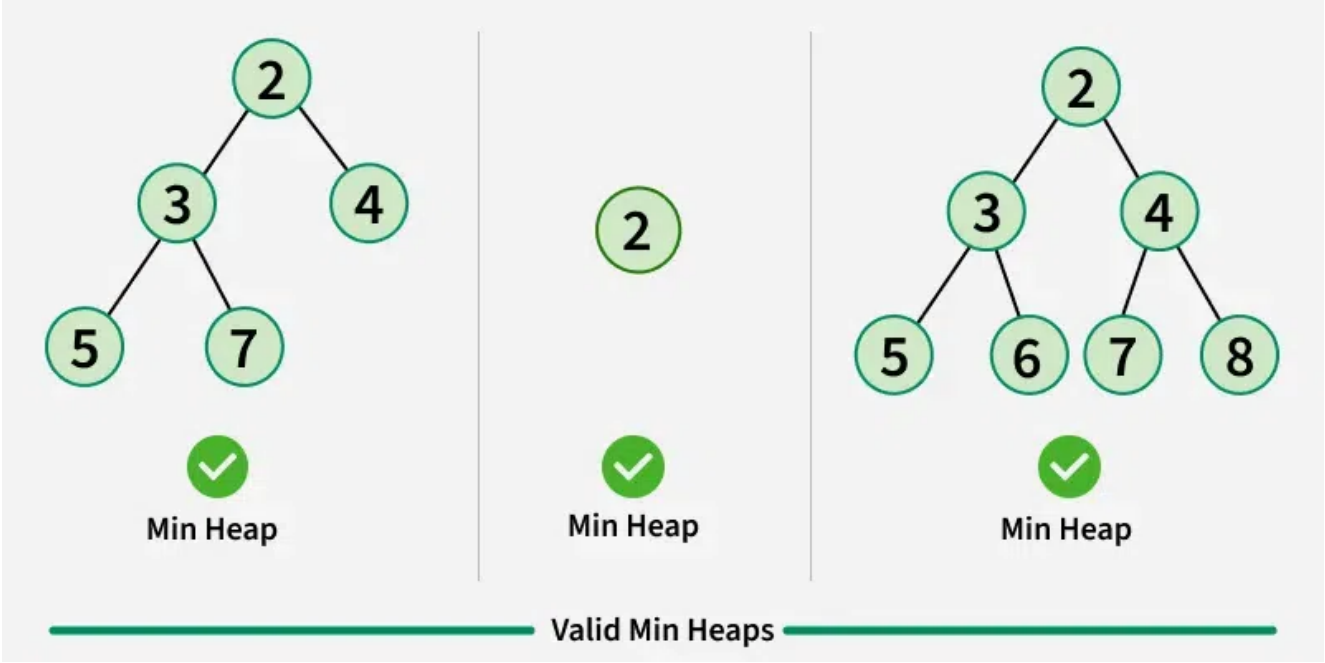
Introduction to Min-Heap – Data Structure and Algorithm Tutorials

Last Updated : 23 Jul, 2025

A **Min-Heap** is a Data Structure with the following properties.

- It is a complete Complete Binary Tree.
- The value of the root node must be the smallest among all its descendant nodes and the same thing must be done for its left and right sub-tree also.





Use Cases of Min-Heap:

- ***Implementing Priority Queue:*** *One of the primary uses of the heap data structure is for implementing priority queues.*
- ***Huffman Coding :*** *Lossless compression algorithm.*
- ***Dijkstra's Algorithm:*** *Dijkstra's algorithm is a shortest path algorithm that finds the shortest path between two nodes in a graph. A min heap can be used to keep track of the unvisited nodes with the smallest distance from the source node.*

- ***Sorting:*** *A min heap can be used as a sorting algorithm to efficiently sort a collection of elements in ascending order.*
- ***Median finding:*** *A min heap can be used to efficiently find the median of a stream of numbers. We can use one min heap to store the larger half of the numbers and one max heap to store the smaller half. The median will be the root of the min heap.*

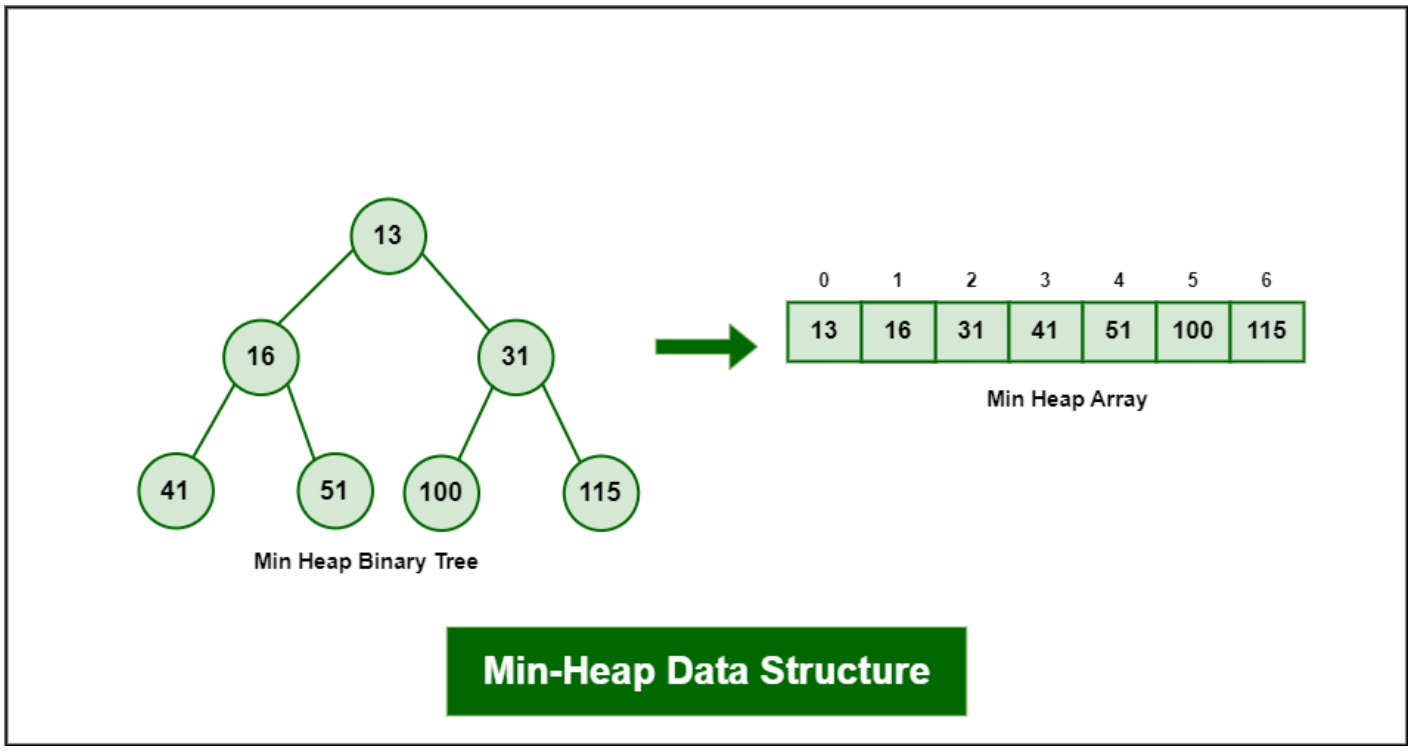
Min-Heap Data structure in Different languages:

- Min-Heap in C++
- Min-Heap in Java
- Min-Heap in Python
- Min-Heap in C# : In C#, a min heap can be implemented using the PriorityQueue<T> class from the System.Collections.Generic namespace.
- Min Heap in JavaScript

C++ Java Python C#

Internal Implementation of Min-Heap Data Structure

Mapping the elements of a heap into an array is trivial: if a node is stored an index k, then its left child is stored at index 2i + 1 and its right child at index 2i + 2.



*A **Min heap is typically represented as an array**.*

- *The root element will be at a**rr[0]**.*
- *For any ith node a**rr[i]**:*
  - ***arr[(i -1) / 2]** returns its parent node.*
  - ***arr[(2 * i) + 1]** returns its left child node.*
  - ***arr[(2 * i) + 2]** returns its right child node.*

The Internal Implementation of the Min-Heap requires 3 major steps:

1. **Insertion**: To insert an element into the min heap, we first append the element to the end of the array and then adjust the heap property by repeatedly swapping the element with its parent until it is in the correct position.
2. **Deletion**: To remove the minimum element from the min heap, we first swap the root node with the last element in the array, remove the last element, and then adjust the heap property by repeatedly swapping the element with its smallest child until it is in the correct position.
3. **Heapify**: A heapify operation can be used to create a min heap from an unsorted array.

Operations on Min-heap Data Structure and their Implementation:

Here are some common operations that can be performed on a Heap Data Structure,

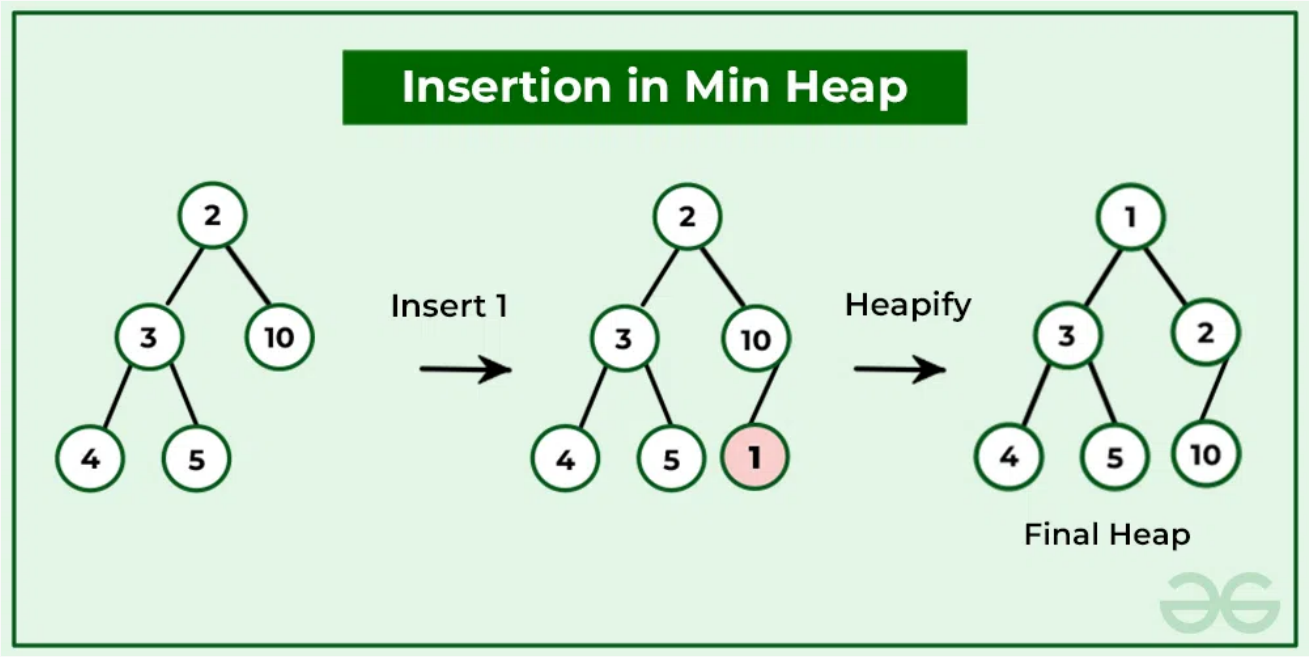
Insertion in Min-Heap Data Structure

Elements can be inserted into the heap following a similar approach as discussed above for deletion. The idea is to:

- The insertion operation in a min-heap involves the following steps:
- Add the new element to the end of the heap, in the next available position in the last level of the tree.
- Compare the new element with its parent. If the parent is greater than the new element, swap them.
- Repeat step 2 until the parent is smaller than or equal to the new element, or until the new element reaches the root of the tree.
- The new element is now in its correct position in the min heap, and the heap property is satisfied.

Illustration:

Suppose the Heap is a Min-Heap as:



Insertion in Min-Heap

Implementation of Insertion Operation in Min-Heap:

C++ Java Python C# JavaScript

```cpp
#include <iostream>
#include <vector>

using namespace std;

// Function to insert a new element into the min-heap
void insert_min_heap(vector<int>& heap, int value)
{
    // Add the new element to the end of the heap
    heap.push_back(value);
    // Get the index of the last element
    int index = heap.size() - 1;
    // Compare the new element with its parent and swap if
    // necessary
    while (index > 0
           && heap[(index - 1) / 2] > heap[index]) {
        swap(heap[index], heap[(index - 1) / 2]);
        // Move up the tree to the parent of the current
        // element
        index = (index - 1) / 2;
    }
}

// Main function to test the insert_min_heap function
int main()
{
    vector<int> heap;
    int values[] = { 10, 7, 11, 5, 4, 13 };
    int n = sizeof(values) / sizeof(values[0]);
    for (int i = 0; i < n; i++) {
        insert_min_heap(heap, values[i]);
        cout << "Inserted " << values[i]
             << " into the min-heap: ";
        for (int j = 0; j < heap.size(); j++) {
            cout << heap[j] << " ";
        }
        cout << endl;
    }
```

```
    return 0;
}
```

Output

```
Inserted 10 into the min-heap: 10
Inserted 7 into the min-heap: 7 10
Inserted 11 into the min-heap: 7 10 11
Inserted 5 into the min-heap: 5 7 11 10
Inserted 4 into the min-heap: 4 5 11 10 7
Inser...
```

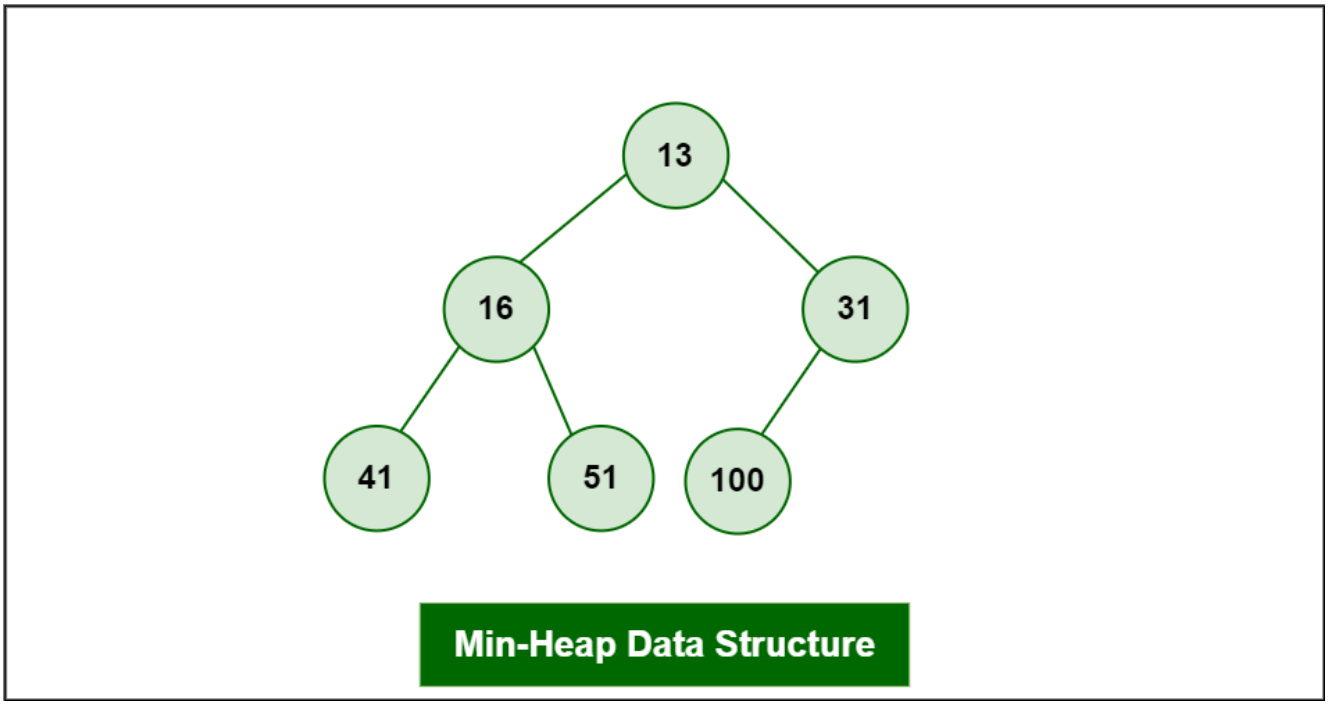
Time Complexity: O(log(n)) (where n is no of elements in the heap**)**
Auxiliary Space: O(n)

Deletion in Min-Heap Data Structure

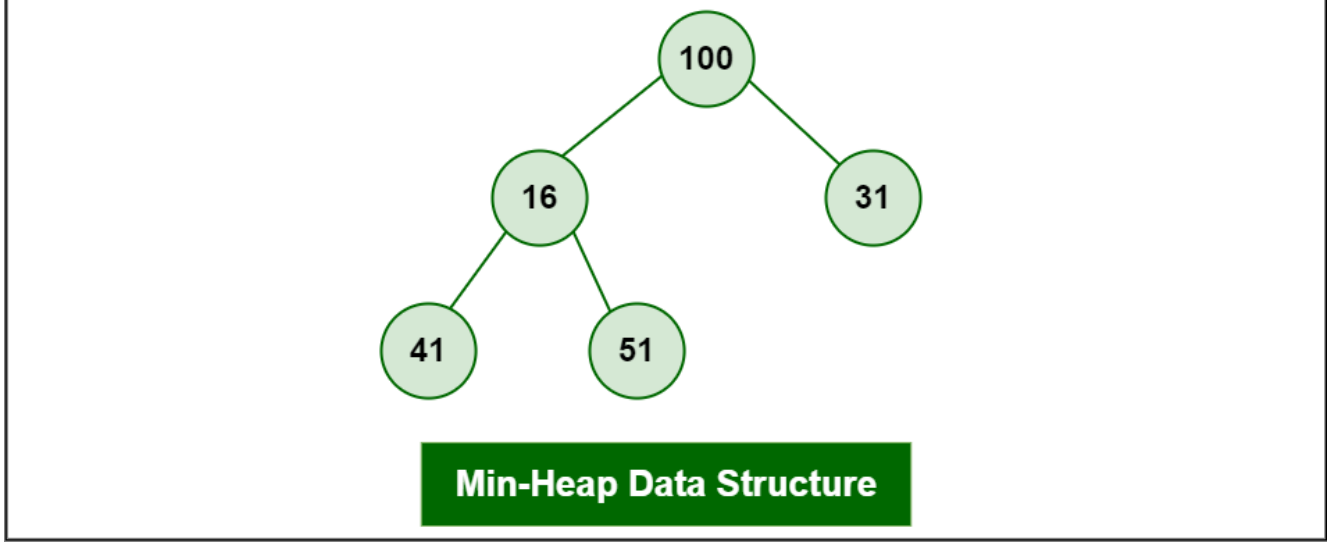
Removing the smallest element (the root) from the min heap. The root is replaced by the last element in the heap, and then the heap property is restored by swapping the new root with its smallest child until the parent is smaller than both children or until the new root reaches a leaf node.

- Replace the root or element to be deleted with the last element.
- Delete the last element from the Heap.
- Since the last element is now placed at the position of the root node. So, it may not follow the heap property. Therefore, heapify the last node placed at the position of the root.

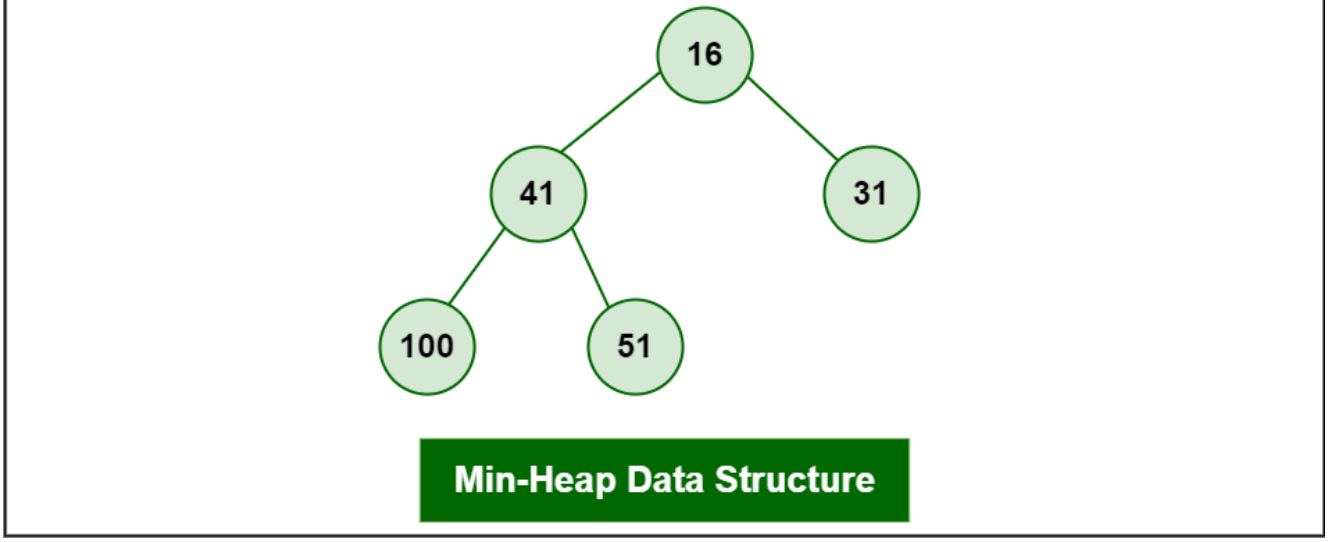
Illustration:



Step 1 Replace the last element with root and delete it.



Step 2 Heapify root



Implementation of Deletion operation in Min-Heap:

C++ Java Python C# JavaScript

```cpp
#include <iostream>
#include <vector>

using namespace std;

// Function to insert a new element into the min-heap
void insert_min_heap(vector<int>& heap, int value)
{
    // Add the new element to the end of the heap
    heap.push_back(value);
    // Get the index of the last element
    int index = heap.size() - 1;
    // Compare the new element with its parent and swap if
    // necessary
    while (index > 0
           && heap[(index - 1) / 2] > heap[index]) {
        swap(heap[index], heap[(index - 1) / 2]);
        // Move up the tree to the parent of the current
        // element
        index = (index - 1) / 2;
    }
}

// Function to delete a node from the min-heap
void delete_min_heap(vector<int>& heap, int value)
{
    // Find the index of the element to be deleted
    int index = -1;
    for (int i = 0; i < heap.size(); i++) {
        if (heap[i] == value) {
            index = i;
            break;
        }
    }
```

```
    // If the element is not found, return
    if (index == -1) {
        return;
    }
    // Replace the element to be deleted with the last
    // element
    heap[index] = heap[heap.size() - 1];
    // Remove the last element
    heap.pop_back();
    // Heapify the tree starting from the element at the
    // deleted index
    while (true) {
        int left_child = 2 * index + 1;
        int right_child = 2 * index + 2;
        int smallest = index;
        if (left_child < heap.size()
            && heap[left_child] < heap[smallest]) {
            smallest = left_child;
        }
        if (right_child < heap.size()
            && heap[right_child] < heap[smallest]) {
            smallest = right_child;
        }
        if (smallest != index) {
            swap(heap[index], heap[smallest]);
            index = smallest;
        }
        else {
            break;
        }
    }
}

// Main function to test the insert_min_heap and
// delete_min_heap functions
int main()
{
    vector<int> heap;
    int values[] = { 13, 16, 31, 41, 51, 100 };
    int n = sizeof(values) / sizeof(values[0]);
    for (int i = 0; i < n; i++) {
        insert_min_heap(heap, values[i]);
```

```
    }
    cout << "Initial heap: ";
    for (int j = 0; j < heap.size(); j++) {
        cout << heap[j] << " ";
    }
    cout << endl;

    delete_min_heap(heap, 13);
    cout << "Heap after deleting 13: ";
    for (int j = 0; j < heap.size(); j++) {
        cout << heap[j] << " ";
    }
    cout << endl;

    return 0;
}
```

Output

```
Initial heap: 13 16 31 41 51 100
Heap after deleting 13: 16 41 31 100 51
```

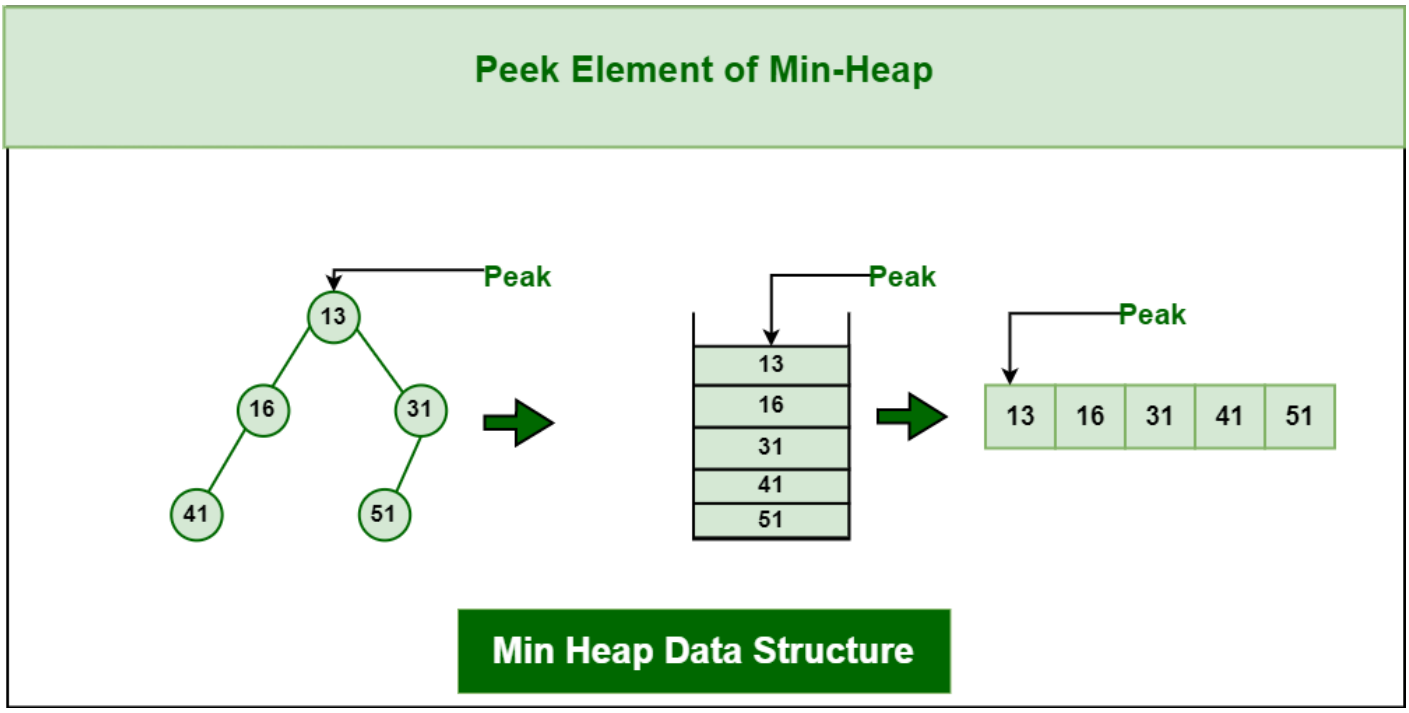
Time complexity: O(log n) where n is no of elements in the heap

Auxiliary Space: O(n)

for more detail refer - Insertion and Deletion in Heaps

Peek operation on Min-Heap Data Structure

To access the minimum element (i.e., the root of the heap), the value of the root node is returned. The time complexity of peek in a min-heap is O(1).



Min Heap Data Structure

Implementation of Peek operation in Min-Heap:

C++ Java Python C# JavaScript

```cpp
#include <iostream>
#include <queue>
#include <vector>
using namespace std;

int main()
{
    // Create a max heap with some elements using a
    // priority_queue
    priority_queue<int, vector<int>, greater<int> > minHeap;
    minHeap.push(9);
    minHeap.push(8);
    minHeap.push(7);
    minHeap.push(6);
    minHeap.push(5);
    minHeap.push(4);
    minHeap.push(3);
    minHeap.push(2);
    minHeap.push(1);

    // Get the peak element (i.e., the largest element)
    int peakElement = minHeap.top();
```

```
    // Print the peak element
    cout << "Peak element: " << peakElement << std::endl;

    return 0;
}
```

Output

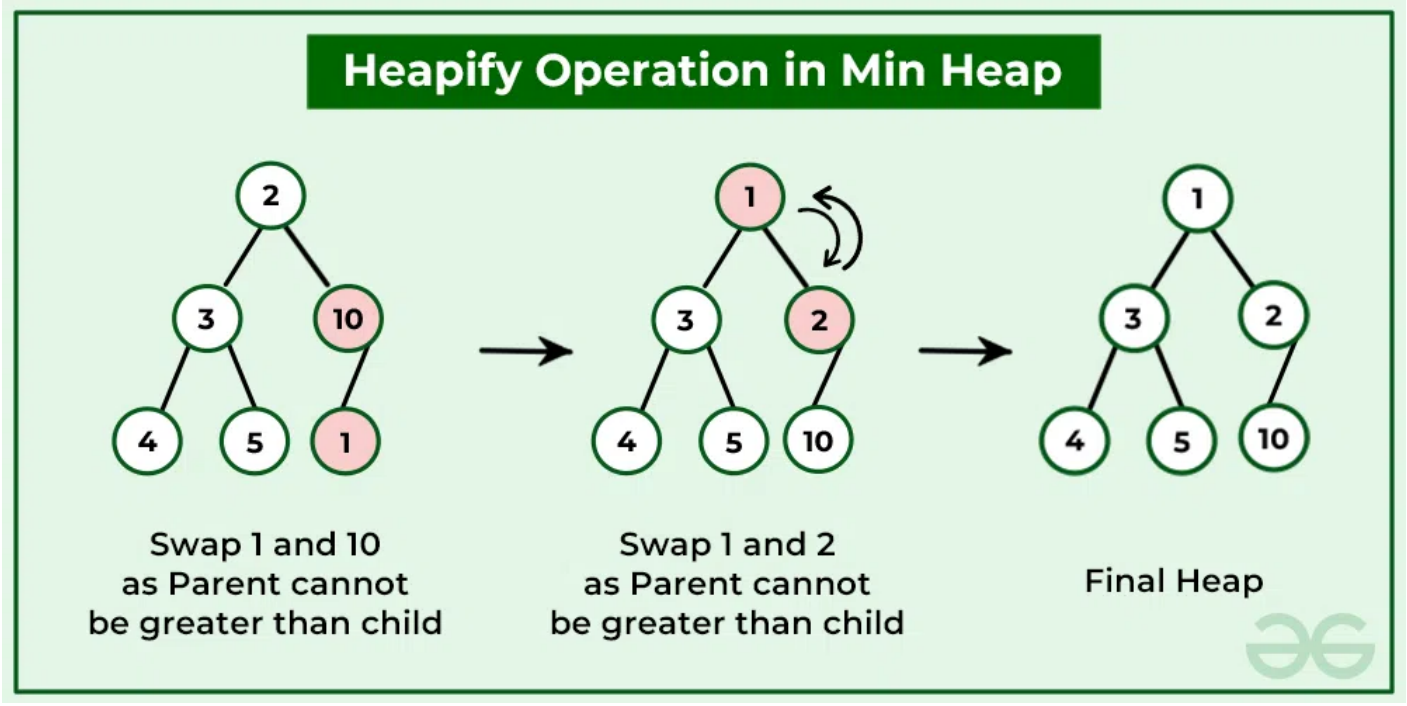
```
Peak element: 1
```

Time complexity: In a min heap implemented using an array or a list, the peak element can be accessed in constant time, O(1), as it is always located at the root of the heap.
In a min heap implemented using a binary tree, the peak element can also be accessed in O(1) time, as it is always located at the root of the tree.

Auxiliary Space: O(n)

Heapify operation on Min-Heap Data Structure

A heapify operation can be used to create a min heap from an unsorted array. This is done by starting at the last non-leaf node and repeatedly performing the "bubble down" operation until all nodes satisfy the heap property.



Heapify operation in Min Heap

Implementation of Heapify operation in Min-Heap:

C++ Java Python C# JavaScript

```cpp
#include <iostream>
#include <vector>
using namespace std;

void minHeapify(vector<int> &arr, int i, int n) {
    int smallest = i;
    int l = 2*i + 1;
    int r = 2*i + 2;

    if (l < n && arr[l] < arr[smallest])
        smallest = l;

    if (r < n && arr[r] < arr[smallest])
        smallest = r;

    if (smallest != i) {
        swap(arr[i], arr[smallest]);
        minHeapify(arr, smallest, n);
    }
}

int main() {
    vector<int> arr = {10, 5, 15, 2, 20, 30};

    cout << "Original array: ";
    for (int i = 0; i < arr.size(); i++)
        cout << arr[i] << " ";

    // Perform heapify operation on min-heap
    for (int i = arr.size()/2 - 1; i >= 0; i--)
        minHeapify(arr, i, arr.size());

    cout << "\nMin-Heap after heapify operation: ";
    for (int i = 0; i < arr.size(); i++)
        cout << arr[i] << " ";

    return 0;
```

```
}
```

Output

```
Original array: 10 5 15 2 20 30
Min-Heap after heapify operation: 2 5 15 10 20 30
```

The time complexity of heapify in a min-heap is O(n).

Search operation on Min-Heap Data Structure

To search for an element in the min heap, a linear search can be performed over the array that represents the heap. However, the time complexity of a linear search is O(n), which is not efficient. Therefore, searching is not a commonly used operation in a min heap.

Here's an example code that shows how to search for an element in a min heap using **std::find()**:

C++ Java Python C# JavaScript

```cpp
#include <bits/stdc++.h>
using namespace std;

int main()
{
    priority_queue<int, vector<int>, greater<int> >
        min_heap;
    // example max heap

    min_heap.push(10);
    min_heap.push(9);
    min_heap.push(8);
    min_heap.push(6);
    min_heap.push(4);

    int element = 6; // element to search for
    bool found = false;

    // Copy the min heap to a temporary queue and search for
    // the element
    std::priority_queue<int, vector<int>, greater<int> >
```

```
        temp = min_heap;
    while (!temp.empty()) {
        if (temp.top() == element) {
            found = true;
            break;
        }
        temp.pop();
    }

    if (found) {
        std::cout << "Element found in the min heap."
                  << std::endl;
    }
    else {
        std::cout << "Element not found in the min heap."
                  << std::endl;
    }

    return 0;
}
```

Output

```
Element found in the min heap.
```

Complexity Analysis

The **time complexity** of this program is **O(n log n)**, where **n** is the number of elements in the priority queue.

The insertion operation has a time complexity of **O(log n)** in the worst case because the heap property needs to be maintained. The search operation involves copying the priority queue to a temporary queue and then traversing the temporary queue, which takes **O(n log n)** time in the worst case because each element needs to be copied and popped from the queue, and the priority queue needs to be rebuilt for each operation.

The **space complexity** of the program is **O(n)** because it stores **n** elements in the priority queue and creates a temporary queue with **n** elements.

Applications of Min-Heap Data Structure

- **Heap sort:** Min heap is used as a key component in heap sort algorithm which is an efficient sorting algorithm with a time complexity of O(nlogn).
- **Priority Queue:** A priority queue can be implemented using a min heap data structure where the element with the minimum value is always at the root.
- **Dijkstra's algorithm:** In Dijkstra's algorithm, a min heap is used to store the vertices of the graph with the minimum distance from the starting vertex. The vertex with the minimum distance is always at the root of the heap.
- **Huffman coding:** In Huffman coding, a min heap is used to implement a priority queue to build an optimal prefix code for a given set of characters.
- **Merge K sorted arrays:** Given K sorted arrays, we can merge them into a single sorted array efficiently using a min heap data structure.

Advantages of Min-heap Data Structure

- **Efficient insertion and deletion**: Min heap allows fast insertion and deletion of elements with a time complexity of O(log n), where n is the number of elements in the heap.
- **Efficient retrieval of minimum element:** The minimum element in a min heap is always at the root of the heap, which can be retrieved in O(1) time.
- **Space efficient:** Min heap is a compact data structure that can be implemented using an array or a binary tree, which makes it space efficient.
- **Sorting:** Min heap can be used to implement an efficient sorting algorithm such as heap sort with a time complexity of O(n log n).
- **Priority Queue:** Min heap can be used to implement a priority queue, where the element with the minimum priority can be retrieved efficiently in O(1) time.
- **Versatility:** Min heap has several applications in computer science, including graph algorithms, data compression, and database systems.

Overall, min heap is a useful and versatile data structure that offers efficient operations, space efficiency, and has several applications in computer science.

Explore

- DSA Fundamentals
- Data Structures
- Algorithms
- Advanced
- Interview Preparation
- Practice Problem